

C++ Standard Library Issues to be moved in Tokyo, Mar. 2024

Doc. no. P3180R0

Date: 2024-03-15

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

- [Ready Issues](#)
- [Tentatively Ready Issues](#)

Ready Issues

[3767](#). `codecvt<charN_t, char8_t, mbstate_t>` incorrectly added to locale

Section: 30.3.1.2.1 [\[locale.category\]](#), 30.4.2.5.1 [\[locale.codecvt.general\]](#) **Status:** [Ready](#) **Submitter:** Victor Zverovich **Opened:** 2022-09-05 **Last modified:** 2023-11-07

Priority: 3

View all other [issues](#) in `[locale.category]`.

Discussion:

Table `[tab:locale.category.facets]` includes the following two facets:

- `codecvt<char16_t, char8_t, mbstate_t>`
- `codecvt<char32_t, char8_t, mbstate_t>`

However, neither of those actually has anything to do with a locale and therefore it doesn't make sense to dynamically register them with `std::locale`. Instead they provide conversions between fixed encodings (UTF-8, UTF-16, UTF-32) that are unrelated to locale encodings other than they may happen to coincide with encodings of some locales by accident.

The issue was introduced when adding `codecvt<char[16|32]_t, char, mbstate_t>` in [N2035](#) which gave no design rationale for using `codecvt` in the first place. Likely it was trying to do a minimal amount of changes and copied the wording for `codecvt<wchar_t, char, mbstate_t>` but unfortunately didn't consider encoding implications.

[P0482](#) changed `char` to `char8_t` in these facets which made the issue more glaring but unfortunately, despite the breaking change, it failed to address it.

Apart from an obvious design mistake this also adds a small overhead for every locale construction because the implementation has to copy these pseudo-facets for no good reason violating "don't pay for what you don't use" principle.

A simple fix is to remove the two facets from table `[tab:locale.category.facets]` and make them directly constructible.

[2022-09-23; Reflector poll]

Set priority to 3 after reflector poll. Send to SG16 (then maybe LEWG).

[2022-09-28; SG16 responds]

SG16 agrees that the `codecvt` facets mentioned in LWG3767 "`codecvt<charN_t, char8_t, mbstate_t>` incorrectly added to locale" are intended to be invariant with respect to locale. Unanimously in favor.

[Issaquah 2023-02-10; LWG issue processing]

Removing these breaks most code using them today, because the most obvious way to use them is via `use_facet` on a locale, which would throw if they're removed (and because they were guaranteed to be present, code using them might have not bothered to check for them using `has_facet`). Instead of removing them, deprecate the guarantee that they're always present (so move them to D.24 [\[depr.locale.category\]](#)). Don't bother changing the destructor. Victor to update wording.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 30.3.1.2.1 [\[locale.category\]](#), Table 105 ([tab:locale.category.facets]) — "Locale category facets" — as indicated:

Table 105: Locale category facets
[tab:locale.category.facets]

Category	Includes facets
	...
ctype	<code>ctype<char>, ctype<wchar_t></code>
	<code>codecvt<char, char, mbstate_t></code>
	<code>codecvt<char16_t, char8_t, mbstate_t></code>
	<code>codecvt<char32_t, char8_t, mbstate_t></code>
	<code>codecvt<wchar_t, char, mbstate_t></code>
	...

2. Modify 30.4.2.5.1 [\[locale.codecvt.general\]](#) as indicated:

```
namespace std {
    [...]
    template<class internT, class externT, class stateT>
    class codecvt : public locale::facet, public codecvt_base {
    public:
        using intern_type = internT;
        using extern_type = externT;
        using state_type = stateT;

        explicit codecvt(size_t refs = 0);
        ~codecvt();

        [...]
    protected:
        ~codecvt();
        [...]
    };
}
```

[...]

-3- The specializations required in Table ~~105 [tab:locale.category.facets]~~¹⁰⁶ [\[tab:locale.spec\]](#) (30.3.1.2.1 [\[locale.category\]](#)) convert the implementation-defined native character set. `codecvt<char, char, mbstate_t>` implements a degenerate conversion; it does not convert at all. The specialization `codecvt<char16_t, char8_t, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specialization `codecvt<char32_t, char8_t, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms. `codecvt<wchar_t, char, mbstate_t>` converts between the native character sets for ordinary and wide characters. Specializations on `mbstate_t` perform conversion between encodings known to the library implementer. Other encodings can be converted by specializing on a program-defined `stateT` type. Objects of type `stateT` can contain any state that is useful to communicate to or from the specialized `do_in` or `do_out` members.

[2023-02-10; Victor Zverovich comments and provides improved wording]

Per today's LWG discussion the following changes have been implemented in revised wording:

- Deprecated the facets instead of removing them (also `_byname` variants which were previously missed).
- Removed the changes to facet dtor since with deprecation it's no longer critical to provide other ways to access them.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 30.3.1.2.1 [\[locale.category\]](#), Table 105 ([tab:locale.category.facets]) — "Locale category facets" — and Table 106 ([tab:locale.spec]) "Required specializations" as indicated:

Table 105: Locale category facets

[tab:locale.category.facets]

Category	Includes facets
...	
ctype	ctype<char>, ctype<wchar_t>
	codecvt<char, char, mbstate_t>
	codecvt<char16_t, char8_t, mbstate_t>
	codecvt<char32_t, char8_t, mbstate_t>
	codecvt<wchar_t, char, mbstate_t>
...	

[...]

Table 106: Required specializations [tab:locale.spec]

Category	Includes facets
...	
ctype	ctype_byname<char>, ctype_byname<wchar_t>
	codecvt_byname<char, char, mbstate_t>
	codecvt_byname<char16_t, char8_t, mbstate_t>
	codecvt_byname<char32_t, char8_t, mbstate_t>
	codecvt_byname<wchar_t, char, mbstate_t>
...	

2. Modify 30.4.2.5.1 [\[locale.codecvt.general\]](#) as indicated:

[...]

-3- The specializations required in Table 105 (30.3.1.2.1 [\[locale.category\]](#)) convert the implementation-defined native character set. `codecvt<char, char, mbstate_t>` implements a degenerate conversion; it does not convert at all. ~~The specialization `codecvt<char16_t, char8_t, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specialization `codecvt<char32_t, char8_t, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms.~~ `codecvt<wchar_t, char, mbstate_t>` converts between the native character sets for ordinary and wide characters. Specializations on `mbstate_t` perform conversion between encodings known to the library implementer. Other encodings can be converted by specializing on a program-defined stateT type. Objects of type stateT can contain any state that is useful to communicate to or from the specialized `do_in` or `do_out` members.

3. Modify D.24 [\[depr.locale.category\]](#) (Deprecated locale category facets) in Annex D as indicated:

-1- The `ctype` locale category includes the following facets as if they were specified in table Table 105 [tab:locale.category.facets] of 30.4.2.5.1 [\[locale.codecvt.general\]](#).

```
codecvt<char16_t, char, mbstate_t>
codecvt<char32_t, char, mbstate_t>
codecvt<char16_t, char8_t, mbstate_t>
codecvt<char32_t, char8_t, mbstate_t>
```

-1- The `ctype` locale category includes the following facets as if they were specified in table Table 106 [tab:locale.spec] of 30.4.2.5.1 [\[locale.codecvt.general\]](#).

```
codecvt_byname<char16_t, char, mbstate_t>
codecvt_byname<char32_t, char, mbstate_t>
codecvt_byname<char16_t, char8_t, mbstate_t>
codecvt_byname<char32_t, char8_t, mbstate_t>
```

-3- The following class template specializations are required in addition to those specified in 30.4.2.5 [\[locale.codecvt\]](#). The specializations `codecvt<char16_t, char, mbstate_t>` and `codecvt<char16_t,`

`char8_t, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specializations `codecvt<char32_t, char, mbstate_t>` and `codecvt<char32_t, char8_t, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms.

3919. `enumerate_view` may invoke UB for sized common non-forward underlying ranges

Section: 26.7.23 [\[range.enumerate\]](#) Status: [Ready](#) Submitter: Patrick Palka Opened: 2023-04-07 Last modified: 2023-11-10

Priority: 3

Discussion:

For a sized common range, `enumerate_view::end()` is specified to call `ranges::distance`. But `ranges::distance` is not necessarily well-defined for a sized non-forward range after calling `ranges::begin` (according to 26.4.3 [\[range.sized\]](#)).

So for a sized common non-forward underlying range, it seems calling `enumerate_view::begin()` followed by `enumerate_view::end()` may invoke UB and thus make `enumerate_view` potentially unusable for such ranges.

I suppose we might need to instead call and cache the result of `ranges::distance` from `enumerate_view::begin()` for such ranges.

[2022-04-12; Patrick Palka provides wording]

The proposed wording follows the suggestion provided by Tim Song, to simply make `enumerate` non-common for this case.

[2023-05-24; Reflector poll]

Set priority to 3 after reflector poll.

[Kona 2023-11-10; move to Ready]

Proposed resolution:

This wording is relative to [N4944](#).

1. Modify 26.7.23.2 [\[range.enumerate.view\]](#), class template class `enumerate_view` synopsis, as indicated:

```
[...]
constexpr auto end() requires (!simple-view<V>) {
    if constexpr (forward_range<V> && common_range<V> && sized_range<V>)
        return iterator<false>(ranges::end(base_), ranges::distance(base_));
    else
        return sentinel<false>(ranges::end(base_));
}
constexpr auto end() const requires range-with-movable-references<const V> {
    if constexpr (forward_range<const V> && common_range<const V> && sized_range<const V>)
        return iterator<true>(ranges::end(base_), ranges::distance(base_));
    else
        return sentinel<true>(ranges::end(base_));
}
[...]
```

3950. `std::basic_string_view` comparison operators are overspecified

Section: 23.3.2 [\[string.view.synop\]](#) Status: [Ready](#) Submitter: Giuseppe D'Angelo Opened: 2023-06-21 Last modified: 2023-11-10

Priority: Not Prioritized

Discussion:

The `<string_view>` synopsis in 23.3.2 [\[string.view.synop\]](#) has these signatures for `operator==` and `operator<=>`:

```
// 23.3.4 \[string.view.comparison\], non-member comparison functions
template<class charT, class traits>
```

```
constexpr bool operator==(basic_string_view<charT, traits> x,
                           basic_string_view<charT, traits> y) noexcept;
template<class charT, class traits>
constexpr see below operator<=>(basic_string_view<charT, traits> x,
                                basic_string_view<charT, traits> y) noexcept;

// see 23.3.4 \[string.view.comparison\], sufficient additional overloads of comparison functions
```

In 23.3.4 [\[string.view.comparison\]](#), paragraph 1 states that "Implementations shall provide sufficient additional overloads" so that all comparisons between a `basic_string_view<C, T>` object and an object of a type convertible to `basic_string_view<C, T>` work (with the reasonable semantics).

The associated Example 1 proposes this implementation strategy for `operator==`:

```
template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> lhs,
                           basic_string_view<charT, traits> rhs) noexcept {
    return lhs.compare(rhs) == 0;
}
template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> lhs,
                           type_identity_t<basic_string_view<charT, traits>> rhs) noexcept {
    return lhs.compare(rhs) == 0;
}
```

With the current semantics of rewritten candidates for the comparison operators, it is however superfluous to actually specify both overloads (the same applies for `operator<=>`).

The second overload (using `type_identity_t`) is indeed necessary to implement the "sufficient additional overloads" part of 23.3.4 [\[string.view.comparison\]](#), but it is also sufficient, as all the following cases

- `sv == sv`
- `sv == convertible_to_sv`
- `convertible_to_sv == sv`

can in fact use it (directly, or after being rewritten e.g. with the arguments swapped).

The reason why we still do have both operators seems to be historical; there is an explanation offered [here](#) by Barry Revzin.

Basically, there were three overloads before a bunch of papers regarding `operator<=>` and `operator==` were merged:

1. `operator==(bsv, bsv)` to deal with `sv == sv`;
2. `operator==(bsv, type_identity_t<bsv>)` and
3. `operator==(type_identity_t<bsv>, bsv)` to deal with `sv == convertible_to_sv` and vice versa.

Overload (1) was necessary because with only (2) and (3) a call like `sv == sv` would otherwise be ambiguous. With the adoption of the rewriting rules, overload (3) has been dropped, without realizing that overload (1) would then become redundant.

The specification of these overloads can be greatly simplified by adjusting the signatures to explicitly use `type_identity_t`.

[Kona 2023-11-10; move to Ready]

Editorial issue [6324](#) provides the changes as a pull request to the draft.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 23.3.2 [\[string.view.synop\]](#), header `<string_view>` synopsis, as indicated:

```
[...]
// 23.3.4 \[string.view.comparison\], non-member comparison functions
template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> x,
                           type_identity_t<basic_string_view<charT, traits>> y) noexcept;
```

```
template<class charT, class traits>
constexpr see below operator<=>(basic_string_view<charT, traits> x,
                                type_identity_t<basic_string_view<charT, traits>> y) noexcept;

// see 23.3.4 [string.view.comparison], sufficient additional overloads of comparison functions
[...]
```

2. Modify 23.3.4 [string.view.comparison] as indicated:

-1- Let S be `basic_string_view<charT, traits>`, and sv be an instance of S . Implementations shall provide sufficient additional overloads marked `constexpr` and `noexcept` so that an object t with an implicit conversion to S can be compared according to Table 81 [tab:string.view.comparison.overloads].

Table 81: Additional
basic_string_view comparison
overloads

[tab:string.view.comparison.overloads]

Expression	Equivalent to
$t == sv$	$S(t) == sv$
$sv == t$	$sv == S(t)$
$t != sv$	$S(t) != sv$
$sv != t$	$sv != S(t)$
$t < sv$	$S(t) < sv$
$sv < t$	$sv < S(t)$
$t > sv$	$S(t) > sv$
$sv > t$	$sv > S(t)$
$t <= sv$	$S(t) <= sv$
$sv <= t$	$sv <= S(t)$
$t >= sv$	$S(t) >= sv$
$sv >= t$	$sv >= S(t)$
$t <=> sv$	$S(t) <=> sv$
$sv <=> t$	$sv <=> S(t)$

[Example 1: A sample conforming implementation for `operator==` would be:

```
template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> lhs,
                           basic_string_view<charT, traits> rhs) noexcept {
    return lhs.compare(rhs) == 0;
}

template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> lhs,
                           type_identity_t<basic_string_view<charT, traits>> rhs) noexcept {
    return lhs.compare(rhs) == 0;
}
```

—end example]

```
template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> lhs,
                           type_identity_t<basic_string_view<charT, traits>> rhs) noexcept;
```

-2- Returns: `lhs.compare(rhs) == 0`.

```
template<class charT, class traits>
constexpr see below operator<=>(basic_string_view<charT, traits> lhs,
                                type_identity_t<basic_string_view<charT, traits>> rhs) noexcept;
```

-3- Let R denote the type `traits::comparison_category` if that *qualified-id* is valid and denotes a type (13.10.3 [temp.deduct]), otherwise R is `weak_ordering`.

-4- Mandates: R denotes a comparison category type (17.11.2 [cmp.categories]).

-5- Returns: `static_cast<R>(lhs.compare(rhs) <=> 0)`.

[Note: The usage of type identity_t as parameter ensures that an object of type basic_string_view<charT, traits> can always be compared with an object of a type T with an implicit conversion to basic_string_view<charT, traits>, and vice versa, as per 12.2.2.3 [over.match.oper]. — end note]

3975. Specializations of basic_format_context should not be permitted

Section: 22.14.6.6 [\[format.context\]](#) Status: [Ready](#) Submitter: Brian Bi Opened: 2023-08-13 Last modified: 2023-11-07

Priority: 3

View all other [issues](#) in `[format.context]`.

Discussion:

The current wording allows users to specialize `std::basic_format_context`. However, an implementation is not likely to accept a program that uses the library in a way that would instantiate such a specialization, because 22.14.6.6 [\[format.context\]](#) does not provide a complete description of the interface that such a specialization would need to have (e.g., it does not provide a means to initialize the exposition-only `args_` member). Since the library was not designed to be able to work with user specializations of `std::basic_format_context`, declaring such specializations should be explicitly disallowed.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4958](#).

1. Modify the 22.14.6.6 [\[format.context\]](#) as indicated:

-1- An instance of `basic_format_context` holds formatting state consisting of the formatting arguments and the output iterator.

-2- The behavior of a program that adds specializations of basic format context is undefined.

-2- Out shall model `output_iterator<const charT&>`.

[2023-09-23; Daniel comments and provides improved wording]

During the reflector discussion, Dietmar pointed out that the constraint can in principle be checked statically (e.g. when the Library creates or refers to an instantiation of `basic_format_context`), so we can reduce the rather draconian consequence of "undefined behaviour" to "ill-formed, no diagnostics required". Furthermore, the new wording also adds the same constraint to `basic_format_parse_context` as suggested by Tim. This is needed, since only one public constructor is specified, but that specification does not allow to construct an object a non-zero `num_args_` or with the type information necessary for the `check_dynamic_spec*` functions, so the library has an unspecified way to realize this.

[2023-10-30; Reflector poll]

Set priority to 3 after reflector poll.

[Kona 2023-11-07; move to Ready]

Proposed resolution:

This wording is relative to [N4958](#).

[Drafting note: The suggested wording form is borrowed from exactly the same wording form used for `allocator_traits`.]

1. Modify 22.14.6.6 [\[format.context\]](#) as indicated:

-1- An instance of `basic_format_context` holds formatting state consisting of the formatting arguments and the output iterator.

-?- If a program declares an explicit or partial specialization of `basic_format_context`, the program is ill-formed, no diagnostic required.

-2- Out shall model `output_iterator<const charT&>`.

2. Modify 22.14.6.5 [\[format.parse.ctx\]](#) as indicated:

-1- An instance of `basic_format_parse_context` holds the format string parsing state consisting of the format string range being parsed and the argument counter for automatic indexing.

-?- If a program declares an explicit or partial specialization of `basic_format_parse_context`, the program is ill-formed, no diagnostic required.

[3984](#). `ranges::to`'s recursion branch may be ill-formed

Section: 26.5.7.2 [\[range.utility.conv.to\]](#) **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-08-23 **Last modified:** 2023-11-07

Priority: 3

View other [active issues](#) in [\[range.utility.conv.to\]](#).

View all other [issues](#) in [\[range.utility.conv.to\]](#).

Discussion:

When `r` is a nested range, `ranges::to` constructs the object recursively through `r | views::transform(...)`.

However, the expression is ill-formed when the type of lvalue `r` does not model `viewable_range` ([demo](#)):

```
#include <ranges>
#include <vector>
#include <list>

int main() {
    std::vector<std::vector<int>>> v;
    auto r = std::views::all(std::move(v));
    auto l = std::ranges::to<std::list<std::list<int>>>>(r); // hard error in MSVC-STL and libc++
}
```

[2023-11-03; Reflector poll]

Set priority to 3 after reflector poll. "Should be `std::forward<R>(r)` instead?"

[Kona 2023-11-07; move to Ready]

Proposed resolution:

This wording is relative to [N4958](#).

1. Modify 26.5.7.2 [\[range.utility.conv.to\]](#) as indicated:

```
template<class C, input_range R, class... Args> requires (!view<C>)
constexpr C to(R&& r, Args&&... args);
```

-1- *Mandates:* `C` is a cv-unqualified class type.

-2- *Returns:* An object of type `C` constructed from the elements of `r` in the following manner:

(2.1) — If `C` does not satisfy `input_range` or `convertible_to<range_reference_t<R>, range_value_t<C>>` is true:

[...]

(2.2) — Otherwise, if `input_range<range_reference_t<R>>` is true:

```
to<C>(ref_view(r) | views::transform([](auto&& elem) {  
    return to<range_value_t<C>>(std::forward<decltype(elem)>(elem));  
}), std::forward<Args>(args)...);
```

(2.3) — Otherwise, the program is ill-formed.

Tentatively Ready Issues

[4011](#). "Effects: Equivalent to return" in `[span.elem]`

Section: 24.7.2.2.6 `[span.elem]` **Status:** [Tentatively Ready](#) **Submitter:** Arthur O'Dwyer **Opened:** 2023-11-09 **Last modified:** 2024-03-11

Priority: Not Prioritized

Discussion:

In reviewing the wording for [P2821](#) `span.at()`, it had been noticed that 24.7.2.2.6 `[span.elem]` uses a lot of "Effects: Equivalent to return [...];" which could be simply "Returns: [...]".

For comparison, 23.3.3.6 `[string_view.access]` uses "Returns: ..." instead, so I suggest that 24.7.2.2.6 `[span.elem]` should be consistent with that.

[2024-03-11; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4964](#).

1. Modify 24.7.2.2.6 `[span.elem]` as indicated:

```
constexpr reference operator[](size_type idx) const;
```

-1- *Preconditions:* `idx < size()` is true.

-2- ~~Effects~~**Returns:** ~~Equivalent to:~~ ~~return~~ `*(data() + idx)` ~~;~~

-?- Throws: Nothing.

```
constexpr reference front() const;
```

-3- *Preconditions:* `empty()` is false.

-4- ~~Effects~~**Returns:** ~~Equivalent to:~~ ~~return~~ `*data()` ~~;~~

-?- Throws: Nothing.

```
constexpr reference back() const;
```

-5- *Preconditions:* `empty()` is false.

-6- ~~Effects~~**Returns:** ~~Equivalent to:~~ ~~return~~ `*(data() + (size() - 1))` ~~;~~

-?- Throws: Nothing.

```
constexpr pointer data() const noexcept;
```

-7- ~~Effects~~**Returns:** ~~Equivalent to:~~ ~~return~~ `data` ~~;~~

[4012](#). `common_view::begin/end` are missing the *simple-view* check

Section: 26.7.19.2 [\[range.common.view\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-11-11 **Last modified:** 2024-03-11

Priority: Not Prioritized

View all other [issues](#) in `[range.common.view]`.

Discussion:

`common_view::begin/end` have exactly the same implementation as their corresponding `const` versions, which implies that when the underlying `V` satisfies *simple-view*, it is sufficient to just provide `const`-qualified members.

[2024-03-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4964](#).

1. Modify 26.7.19.2 [\[range.common.view\]](#) as indicated:

```
namespace std::ranges {
    template<view V>
        requires (!common_range<V> && copyable<iterator_t<V>>())
        class common_view : public view_interface<common_view<V>> {
        private:
            V base_ = V(); // exposition only

        public:
            [...]
            constexpr auto begin() requires (!simple-view<V>) {
                if constexpr (random_access_range<V> && sized_range<V>)
                    return ranges::begin(base_);
                else
                    return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::begin(base_));
            }

            constexpr auto begin() const requires range<const V> {
                if constexpr (random_access_range<const V> && sized_range<const V>)
                    return ranges::begin(base_);
                else
                    return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::begin(base_));
            }

            constexpr auto end() requires (!simple-view<V>) {
                if constexpr (random_access_range<V> && sized_range<V>)
                    return ranges::begin(base_) + ranges::distance(base_);
                else
                    return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::end(base_));
            }

            constexpr auto end() const requires range<const V> {
                if constexpr (random_access_range<const V> && sized_range<const V>)
                    return ranges::begin(base_) + ranges::distance(base_);
                else
                    return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::end(base_));
            }

            [...]
        };
    [...]
}
```

[4013](#). `lazy_split_view::outer_iterator::value_type` should not provide default constructor

Section: 26.7.16.4 [\[range.lazy.split.outer.value\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-11-11 **Last modified:** 2024-03-12

Priority: Not Prioritized

View all other [issues](#) in [\[range.lazy.split.outer.value\]](#).

Discussion:

After [P2325](#), there is no reason for `lazy_split_view::outer-iterator::value_type` to provide a default constructor, which only leads to [unexpected behavior](#):

```
#include <ranges>

constexpr int arr[] = {42};
constexpr auto split = arr | std::views::lazy_split(0);
static_assert(!std::ranges::range_value_t<decltype(split)>{}); // UB, dereferencing a null pointer
```

Also, the other constructor should be private because it makes no sense for the user to construct it arbitrarily, which is not the intention.

[2024-03-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4964](#).

1. Modify 26.7.16.4 [\[range.lazy.split.outer.value\]](#), class `split_view::outer-iterator::value_type` synopsis, as indicated:

```
namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V> && view<Pattern> &&
            indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
                (forward_range<V> || tiny_range<Pattern>)
    template<bool Const>
    struct lazy_split_view<V, Pattern>::outer-iterator<Const>::value_type
        : view_interface<value_type> {
    private:
        outer-iterator i_ = outer-iterator(); // exposition only

        constexpr explicit value_type(outer-iterator i); // exposition only

    public:
        value_type() = default;
        constexpr explicit value_type(outer-iterator i);

        constexpr inner-iterator<Const> begin() const;
        constexpr default_sentinel_t end() const noexcept;
    };
}
```

[4016](#). *container-insertable* checks do not match what *container-inserter* does

Section: 26.5.7 [\[range.utility.conv\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2023-11-24 **Last modified:** 2024-03-11

Priority: Not Prioritized

Discussion:

The exposition-only helper *container-inserter* uses either `std::back_inserter` or `std::inserter`. Both `std::back_insert_iterator` and `std::insert_iterator` require `C::value_type` to be a valid type, and we do not check for that in *container-insertable*. The insert iterators can also incur a conversion to construct a `C::value_type` which then gets moved into the container. Using `emplace` instead of `insert` would avoid that temporary object. It's also possible (although arguably not worth caring about) that `range_value_t<C>` is not the same type as `C::value_type`, and that conversion to `C::value_type` could be ill-formed (we only check that conversion from `range_reference_t<R>` to `range_value_t<C>` is well-formed).

It seems preferable to remove the use of insert iterators, so that we don't need to check their requirements at all.

[2023-1-26; Rename exposition-only concept and function after reflector discussion.]

[2024-03-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4964](#).

1. Modify 26.5.7.1 [\[range.utility.conv.general\]](#) as indicated:

-4- Let `container-insertableappendable` be defined as follows:

```
template<class Container, class Ref>
constexpr bool container-insertableappendable =           // exposition only
    requires(Container& c, Ref&& ref) {
        requires { c.emplace_back(std::forward<Ref>(ref)); } ||
        requires { c.push_back(std::forward<Ref>(ref)); } ||
        requires { c.emplace(c.end(), std::forward<Ref>(ref)); } ||
        requires { c.insert(c.end(), std::forward<Ref>(ref)); }
    };
```

-5- Let `container-inserterappend` be defined as follows:

```
template<class Container, class Ref>
constexpr auto container-inserterappend(Container& c) {           // exposition only
    if constexpr (requires { c.push_back(declval<Ref>()); })
        return back_inserter(c);
    else
        return inserter(c, c.end());
    return [&c]<class Ref>(Ref&& ref) {
        if constexpr (requires { c.emplace_back(declval<Ref>()); })
            c.emplace_back(std::forward<Ref>(ref));
        else if constexpr (requires { c.push_back(declval<Ref>()); })
            c.push_back(std::forward<Ref>(ref));
        else if constexpr (requires { c.emplace(c.end(), declval<Ref>()); })
            c.emplace(c.end(), std::forward<Ref>(ref));
        else
            c.insert(c.end(), std::forward<Ref>(ref));
    };
};
```

2. Modify 26.5.7.2 [\[range.utility.conv.to\]](#) as indicated:

(2.1.4) Otherwise, if

- `constructible_from<C, Args...>` is true, and
- `container-insertableappendable<C, range_reference_t<R>>` is true:

```
C c(std::forward<Args>(args)...);
if constexpr (sized_range<R> && reservable_container<C>)
    c.reserve(static_cast<range_size_t<C>>(ranges::size(r)));
ranges::copy_for_each(r, container-inserterappend<range_reference_t<R>>(c));
```

4023. Preconditions of `std::basic_streambuf::setg/setp`

Section: 31.6.3.4 [\[streambuf.protected\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2023-12-08 **Last modified:** 2024-03-11

Priority: Not Prioritized

Discussion:

It seems that operations of `std::basic_streambuf` expect that

- `[eback(), egptr())` is a valid range and `gptr()` points into that range, and
- `[pbase(), pptr())` is a valid range and `eptr()` points into that range.

However, it is currently not specified for `setg/setp` that such invariants need to be established.

[2024-03-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4964](#).

1. Modify 31.6.3.4.2 [\[streambuf.get.area\]](#) as indicated:

```
void setg(char_type* gbeg, char_type* gnext, char_type* gend);
```

-?- Preconditions: `[gbeg, gnext)`, `[gbeg, gend)`, and `[gnext, gend)` are all valid ranges.

-?- Postconditions: `gbeg == eback()`, `gnext == gptr()`, and `gend == egptr()` are all true.

2. Modify 31.6.3.4.3 [\[streambuf.put.area\]](#) as indicated:

```
void setp(char_type* pbeg, char_type* pend);
```

-?- Preconditions: `[pbeg, pend)` is a valid range.

-?- Postconditions: `pbeg == pbase()`, `pbeg == pptr()`, and `pend == eptr()` are all true.

4025. Move assignment operator of `std::expected<cv void, E>` should not be conditionally deleted

Section: 22.8.7.4 [\[expected.void.assign\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2023-12-16 **Last modified:** 2024-03-11

Priority: Not Prioritized

View all other [issues](#) in `[expected.void.assign]`.

Discussion:

It seems intended that copy functions of `std::optional`, `std::variant`, and `std::expected` are conditionally deleted, while move functions are constrained. However, the move assignment operator of `std::expected<cv void, E>` is currently conditionally deleted, which is inconsistent.

[2024-03-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 22.8.7.4 [\[expected.void.assign\]](#) as indicated:

```
constexpr expected& operator=(expected&& rhs) noexcept(see below);
```

-?- Constraints: `is_move_constructible_v<E>` is true and `is_move_assignable_v<E>` is true.

[...]

-?- Remarks: The exception specification is equivalent to `is_nothrow_move_constructible_v<E>` && `is_nothrow_move_assignable_v<E>`.

~~-7- This operator is defined as deleted unless is_move_constructible_v<E> is true and is_move_assignable_v<E> is true.~~

4030. Clarify whether arithmetic expressions in [numeric.sat.func] are mathematical or C++

Section: 27.10.17.1 [numeric.sat.func] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe **Opened:** 2023-12-18 **Last modified:** 2024-03-11

Priority: Not Prioritized

Discussion:

During the application of [P0543R0](#), "Saturation arithmetic", it was pointed out that it might not be entirely clear what we want something like " $x + y$ " to mean. The paper does not suggest any formatting for those symbols, and a non-normative note explains that the intention is for the expression to be considered mathematically.

I wonder if this is clear enough. Notes are not normative, and the document "shall be usable without notes". I have formatted this as `$x + y$` throughout, i.e. the variables are in code font, but the symbol is maths, not code. This is quite subtle. (See also [GitHub discussion](#).)

I think it would be an improvement if we simply made the note not be a note. It seems to contain entirely reasonable, mandatory content.

[2024-03-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 27.10.17.1 [numeric.sat.func] as indicated:

-1- ~~Note 1:~~ In the following descriptions, an arithmetic operation is performed as a mathematical operation with infinite range and then it is determined whether the mathematical result fits into the result type. ~~—end note~~

4031. bad_expected_access<void> member functions should be noexcept

Section: 22.8.5 [expected.bad.void] **Status:** [Tentatively Ready](#) **Submitter:** Cassio Neri **Opened:** 2023-12-24 **Last modified:** 2024-03-12

Priority: Not Prioritized

Discussion:

According to 17.9.3 [exception]/2:

Each standard library class T that derives from class `exception` has the following publicly accessible member functions, each of them having a non-throwing exception specification (14.5):

(2.1) — default constructor (unless the class synopsis shows other constructors)

(2.2) — copy constructor

(2.3) — copy assignment operator

For good reasons, `bad_expected_access<void>` overrules from this general rule by protecting its special member functions. However, there's no reason these functions should not be `noexcept`.

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 22.8.5 [\[expected.bad.void\]](#) as indicated:

```
namespace std {
    template<>
    class bad_expected_access<void> : public exception {
    protected:
        bad_expected_access() noexcept;
        bad_expected_access(const bad_expected_access&) noexcept;
        bad_expected_access(bad_expected_access&&) noexcept;
        bad_expected_access& operator=(const bad_expected_access&) noexcept;
        bad_expected_access& operator=(bad_expected_access&&) noexcept;
        ~bad_expected_access();
    public:
        const char* what() const noexcept override;
    };
}
```

[4035](#). `single_view` should provide empty

Section: 26.6.3.2 [\[range.single.view\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-12-31 **Last modified:** 2024-03-12

Priority: Not Prioritized

View all other [issues](#) in `[range.single.view]`.

Discussion:

Although `single_view::empty` can be synthesized through `view_interface`, it seems more worthwhile to provide a static empty for it which eliminates the need to pass in an object parameter, guarantees noexcept-ness, and is consistent with the design of `empty_view` ([demo](#)):

```
#include <ranges>
auto empty = std::views::empty<int>;
static_assert(noexcept(empty.empty()));
static_assert(noexcept(empty.size()));
auto single = std::views::single(0);
static_assert(noexcept(single.empty())); // fire
static_assert(noexcept(single.size()));
```

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 26.6.3.2 [\[range.single.view\]](#) as indicated:

```
namespace std::ranges {
    template<move_constructible T>
        requires is_object_v<T>
    class single_view : public view_interface<single_view<T>> {
    [...]
    public:
        [...]
        constexpr T* begin() noexcept;
        constexpr const T* begin() const noexcept;
        constexpr T* end() noexcept;
        constexpr const T* end() const noexcept;
        static constexpr bool empty() noexcept;
        static constexpr size_t size() noexcept;
        constexpr T* data() noexcept;
        constexpr const T* data() const noexcept;
```

```

    };
    [...]
}

[...]

constexpr T* end() noexcept;
constexpr const T* end() const noexcept;

-5- Effects: Equivalent to: return data() + 1;

static constexpr bool empty() noexcept;

-?- Effects: Equivalent to: return false;

```

4036. `__alignof_is_defined` is only implicitly specified in C++ and not yet deprecated

Section: D.11 [\[depr.c.macros\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2024-01-12 **Last modified:** 2024-03-12

Priority: Not Prioritized

Discussion:

Currently 17.14.4 [\[stdalign.h.syn\]](#) states

The contents of the C++ header `<stdalign.h>` are the same as the C standard library header `<stdalign.h>`, with the following changes: The header `<stdalign.h>` does not define a macro named `alignas`.

SEE ALSO: ISO/IEC 9899:2018, 7.15

which implicitly specifies that `__alignof_is_defined` is also provided in C++, because C17 specified that the macro is provided in `<stdalign.h>`.

However, there's no explicit mentioning of `__alignof_is_defined` in the C++ standard wording. And D.11 [\[depr.c.macros\]](#)/1 (added by LWG [3827](#)) seemingly contradicts with 17.14.4 [\[stdalign.h.syn\]](#) and only makes `__alignas_is_defined` deprecated.

It seems that we should explicitly mention `__alignof_is_defined` in D.11 [\[depr.c.macros\]](#) at this moment.

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify D.11 [\[depr.c.macros\]](#) as indicated:

-1- The header `<stdalign.h>` has the following macros:

```

#define __alignas_is_defined 1
#define __alignof_is_defined 1

```

4037. Static data members of `ctype_base` are not yet required to be usable in constant expressions

Section: 30.4.2.1 [\[category.ctype.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2024-01-12 **Last modified:** 2024-01-14

Priority: Not Prioritized

Discussion:

It may be desired that static data members `ctype_base` are "real constants", i.e. usable in constant expressions. However, this is not strictly required because `mask` is only required to be a bitmask type that can be a class type, which makes the plain `const`

potentially insufficient.

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 30.4.2.1 [\[category.ctype.general\]](#) as indicated:

```
namespace std {
    class ctype_base {
    public:
        using mask = see below;

        // numeric values are for exposition only.
        static constexpr mask space = 1 << 0;
        static constexpr mask print = 1 << 1;
        static constexpr mask cntrl = 1 << 2;
        static constexpr mask upper = 1 << 3;
        static constexpr mask lower = 1 << 4;
        static constexpr mask alpha = 1 << 5;
        static constexpr mask digit = 1 << 6;
        static constexpr mask punct = 1 << 7;
        static constexpr mask xdigit = 1 << 8;
        static constexpr mask blank = 1 << 9;
        static constexpr mask alnum = alpha | digit;
        static constexpr mask graph = alnum | punct;
    };
}
```

- 1- The type mask is a bitmask type (16.3.3.3.3 [\[bitmask.types\]](#)).

4038. std::text_encoding::aliases_view should have constexpr iterators

Section: 30.6.2.5 [\[text.encoding.aliases\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2024-01-16 **Last modified:** 2024-03-12

Priority: Not Prioritized

Discussion:

`aliases_view::begin()` and `aliases_view::end()` are `constexpr` functions, but there is no requirement that you can use the returned iterator and sentinel in constant expressions.

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 30.6.2.5 [\[text.encoding.aliases\]](#) as indicated:

```
struct text_encoding::aliases_view : ranges::view_interface<text_encoding::aliases_view> {
    constexpr implementation-defined begin() const;
    constexpr implementation-defined end() const;
};
```

- 1- `text_encoding::aliases_view` models copyable, `ranges::view`, `ranges::random_access_range`, and `ranges::borrowed_range`.

- 2- Both `ranges::range_value_t<text_encoding::aliases_view>` and `ranges::range_reference_t<text_encoding::aliases_view>` denote `const char*`.

4043. "ASCII" is not a registered character encoding

Section: 30.6.2.2 [\[text.encoding.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2024-01-23 **Last modified:** 2024-03-12

Priority: Not Prioritized

Discussion:

The IANA Character Sets registry does not contain "ASCII" as an alias of the "US-ASCII" encoding. This is apparently for historical reasons, because there used to be some ambiguity about exactly what "ASCII" meant. I don't think those historical reasons are relevant to C++26, but the absence of "ASCII" in the IANA registry means that it's not a registered character encoding as defined by 30.6.2.2 [\[text.encoding.general\]](#).

This means that the encoding referred to by notes in the C++ standard (31.12.6.2 [\[fs.path.generic\]](#), 30.4.4.1.3 [\[facet.numpunct.virtuals\]](#)) and by an example in the `std::text_encoding` proposal ([P1885](#)) isn't actually usable in portable code. So `std::text_encoding("ASCII")` creates an object with `mib() == std::text_encoding::other`, which is not the same encoding as `std::text_encoding("US-ASCII")`. This seems surprising.

[2024-03-12; Reflector poll]

SG16 approved the proposed resolution. Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 30.6.2.2 [\[text.encoding.general\]](#) as indicated:

-1- A *registered character encoding* is a character encoding scheme in the IANA Character Sets registry.

[*Note 1:* The IANA Character Sets registry uses the term “character sets” to refer to character encodings. — *end note*]

The primary name of a registered character encoding is the name of that encoding specified in the IANA Character Sets registry.

-2- The set of known registered character encodings contains every registered character encoding specified in the IANA Character Sets registry except for the following:

(2.1) – NATS-DANO (33)

(2.2) – NATS-DANO-ADD (34)

-3- Each known registered character encoding is identified by an enumerator in `text_encoding::id`, and has a set of zero or more *aliases*.

-4- The set of aliases of a known registered character encoding is an implementation-defined superset of the aliases specified in the IANA Character Sets registry. [The set of aliases for US-ASCII includes "ASCII"](#). No two aliases or primary names of distinct registered character encodings are equivalent when compared by `text_encoding::comp-name`.

4045. tuple can create dangling references from *tuple-like*

Section: 22.4.4.2 [\[tuple.cnstr\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2024-01-24 **Last modified:** 2024-03-12

Priority: Not Prioritized

View other [active issues](#) in `[tuple.cnstr]`.

View all other [issues](#) in [tuple.cnstr].

Discussion:

[P2165R4](#) (Compatibility between tuple, pair and tuple-like objects) added two new constructors to `std::tuple`:

```
template<tuple-like UTuple>
constexpr explicit(see below) tuple(UTuple&& u);
```

and the allocator-extended equivalent. Unlike the existing constructors taking a single parameter of tuple type, these new constructors are not defined as deleted if they would create a dangling reference to a temporary. The existing constructors gained that restriction from [P2255R2](#) (A type trait to detect reference binding to temporary) which was approved one meeting before [P2165R4](#) so LWG seem to have missed the inconsistency.

The proposal also added a new constructor for `std::pair`:

```
template<pair-like P> constexpr explicit(see below) pair(P&& p);
```

This *is* deleted if it would create a dangling reference, although that seems to be an almost accidental consequence of adding the new signature after existing ones which already have the *Remarks*: about being deleted.

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after eleven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 22.4.4.2 [\[tuple.cnstr\]](#) as indicated:

```
template<tuple-like UTuple>
constexpr explicit(see below) tuple(UTuple&& u);
```

-28- Let I be the pack $0, 1, \dots, (\text{sizeof} \dots (\text{Types}) - 1)$.

-29- *Constraints*:

- (29.1) – *different-from*<UTuple, tuple> (26.5.2 [\[range.utility.helpers\]](#)) is true,
- (29.2) – `remove_cvref_t<UTuple>` is not a specialization of `ranges::subrange`,
- (29.3) – `sizeof... (Types)` equals `tuple_size_v<remove_cvref_t<UTuple>>`,
- (29.4) – `(is_constructible_v<Types, decltype(get<I>(std::forward<UTuple>(u)))> && ...)` is true, and
- (29.5) – either `sizeof... (Types)` is not 1, or (when `Types... expands to T`) `is_convertible_v<UTuple, T>` and `is_constructible_v<T, UTuple>` are both false.

-30- *Effects*: For all i , initializes the i^{th} element of `*this` with `get<i>(std::forward<UTuple>(u))`.

-31- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(is_convertible_v<decltype(get<I>(std::forward<UTuple>(u))), Types> && ...)
```

The constructor is defined as deleted if

`(reference_constructs_from_temporary_v<Types, decltype(get<I>(std::forward<UTuple>(u)))> | | ...)`
is true.

[4053](#). Unary call to `std::views::repeat` does not decay the argument

Section: 26.6.5.2 [\[range.repeat.view\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2024-02-05 **Last modified:** 2024-03-12

Priority: Not Prioritized

View other [active issues](#) in [range.repeat.view].

View all other [issues](#) in [range.repeat.view].

Discussion:

Currently, a binary call to `std::views::repeat` decay the arguments due to the deduction guide, but a unary call doesn't, which is inconsistent.

For example:

```
#include <concepts>
#include <ranges>

using RPV = std::ranges::repeat_view<const char*>;

static_assert(std::same_as<decltype(std::views::repeat("foo", std::unreachable_sentinel)), RPV>); // OK
static_assert(std::same_as<decltype(std::views::repeat("+foo", std::unreachable_sentinel)), RPV>); // OK
static_assert(std::same_as<decltype(std::views::repeat("foo")), RPV>); // ill-formed
static_assert(std::same_as<decltype(std::views::repeat("+foo")), RPV>); // OK
```

Presumably we should extend the deduction guide of `std::ranges::repeat_view` to cover the unary cases.

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 26.6.5.2 [\[range.repeat.view\]](#), class template `repeat_view` synopsis, as indicated:

[Drafting note: The proposed wording has been suggested by Casey Carter, see [microsoft/STL#3576](#)]

```
namespace std::ranges {
    [...]

    template<class T, class Bound = unreachable_sentinel_t>
        repeat_view(T, Bound = Bound()) -> repeat_view<T, Bound>;
}
```

[4054](#). Repeating a `repeat_view` should repeat the view

Section: 26.6.5.1 [\[range.repeat.overview\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2024-02-12 **Last modified:** 2024-03-12

Priority: Not Prioritized

Discussion:

`views::repeat(views::repeat(5))` should be a view of `repeat_views`, but it's currently a view of `ints` due to the use of CTAD in the specification of `views::repeat`.

[2024-03-12; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 26.6.5.1 [\[range.repeat.overview\]](#) as indicated:

-1- `repeat_view` generates a sequence of elements by repeatedly producing the same value.

-2- The name `views::repeat` denotes a customization point object (16.3.3.3.5 [\[customization.point.object\]](#)). Given subexpressions `E` and `F`, the expressions `views::repeat(E)` and `views::repeat(E, F)` are expression-equivalent to `repeat_view<decay_t<decltype((E))>>(E)` and `repeat_view(E, F)`, respectively.